

05_alert-detail-js

#javascript #frontend #api #fastapi #html #SIEM #SOC

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ frontend/  
   └─ assets/  
      └─ alert_detail.js
```

El archivo `alert_detail.js` se encuentra dentro de:

```
frontend/assets/
```

Este archivo contiene la lógica JavaScript específica de la página de detalle de alerta:

```
frontend/alert.html
```

Su función es cargar una alerta concreta desde la API, mostrar sus datos en pantalla y permitir actualizar su estado mediante botones.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,360p' frontend/assets/alert_detail.js
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Permite visualizar contenido de archivos.

```
-n
```

Evita que `sed` imprima todo automáticamente.

```
'1,360p'
```

Imprime desde la línea 1 hasta la 360.

```
frontend/assets/alert_detail.js
```

Ruta del archivo analizado.

3 Código completo del archivo

```
let alertId = null;  
let currentAlert = null;  
  
function setButtonsForStatus(status) {  
  const btnAck = qs("btnAck");  
  const btnClose = qs("btnClose");  
  const btnReopen = qs("btnReopen");
```

```

    btnAck.disabled = (status !== "open");
    btnClose.disabled = (status === "closed");
    btnReopen.disabled = (status !== "ack" && status !== "closed");
}

function renderAlert(a) {
    currentAlert = a;

    qs("title").textContent = `Alerta #${a.id}`;
    qs("subtitle").textContent = `status=${a.status} · group_key=${a.group_key ?? "-"}`;

    qs("v_id").textContent = a.id;
    qs("v_status").textContent = a.status;
    qs("v_status").className = statusBadgeClass(a.status);
    qs("v_group_key").textContent = a.group_key ?? "-";
    qs("v_rule_id").textContent = a.rule_id;
    qs("v_event_id").textContent = a.event_id;
    qs("v_created_at").textContent = formatDate(a.created_at);
    qs("v_updated_at").textContent = formatDate(a.updated_at);
    qs("v_title").textContent = a.title;

    setButtonsForStatus(a.status);
}

async function loadAlert() {
    const err = qs("errorBox");
    const info = qs("infoBox");
    hide(err); hide(info);

    if (!alertId) {
        show(err, "Falta parámetro ?id= en la URL.");
        return;
    }

    try {
        const a = await apiFetch(`/alerts/${alertId}`);
        renderAlert(a);
    } catch (e) {
        show(err, `Error cargando alerta: ${e.message}`);
    }
}

async function updateStatus(nextStatus) {
    const err = qs("errorBox");
    const info = qs("infoBox");
    hide(err); hide(info);

    try {
        const a = await apiFetch(`/alerts/${alertId}`, {
            method: "PATCH",
            body: { status: nextStatus }
        });
        renderAlert(a);
        show(info, `Estado actualizado a: ${a.status}`);
    } catch (e) {
        show(err, `Error actualizando estado: ${e.message}`);
    }
}

function init() {
    alertId = getQueryParam("id");

    qs("btnAck").addEventListener("click", () => updateStatus("ack"));
    qs("btnClose").addEventListener("click", () => updateStatus("closed"));
}

```

```
qs("btnReopen").addEventListener("click", () => updateStatus("open"));

loadAlert();
}

document.addEventListener("DOMContentLoaded", init);
```

4 Función general del archivo

`alert_detail.js` controla la página de detalle de alerta.

Su papel es convertir `alert.html` en una vista dinámica.

Responsabilidades principales:

- Leer el parámetro `id` desde la URL.
- Consultar la alerta concreta con `GET /alerts/{id}`.
- Renderizar los datos de la alerta en el HTML.
- Activar o desactivar botones según el estado actual.
- Actualizar el estado con `PATCH /alerts/{id}`.
- Mostrar mensajes de error o confirmación.

La relación principal es:

```
alert.html
  ↓
define estructura visual

alert_detail.js
  ↓
lee id
  ↓
consulta backend
  ↓
rellena campos
  ↓
permite cambiar estado
```

5 Estructura general del archivo

El archivo puede dividirse en seis bloques:

```
let alertId = null;
let currentAlert = null;
```

Variables globales de estado.

```
function setButtonsForStatus(status) { ... }
```

Controla qué botones están habilitados según el estado de la alerta.

```
function renderAlert(a) { ... }
```

Pinta la alerta en la página.

```
async function loadAlert() { ... }
```

Carga la alerta desde la API.

```
async function updateStatus(nextStatus) { ... }
```

Actualiza el estado de la alerta.

```
function init() { ... }  
document.addEventListener("DOMContentLoaded", init);
```

Inicializa la página cuando el DOM está listo.

Visualmente:

```
alert_detail.js  
├─ alertId  
├─ currentAlert  
├─ setButtonsForStatus()  
├─ renderAlert()  
├─ loadAlert()  
├─ updateStatus()  
├─ init()  
└─ DOMContentLoaded
```

6 Análisis línea por línea

Variable `alertId`

```
let alertId = null;
```

Declara una variable global llamada `alertId`.

Inicialmente vale `null`.

Después se rellenará con el parámetro `id` de la URL.

Ejemplo:

```
alert.html?id=1
```

En ese caso:

```
alertId = "1";
```

Esta variable es necesaria para saber qué alerta consultar y actualizar.

Variable `currentAlert`

```
let currentAlert = null;
```

Declara una variable global llamada `currentAlert`.

Inicialmente vale `null`.

Después se usará para guardar la alerta cargada desde la API.

Aunque en el código actual no se usa mucho más allá de almacenarla, puede ser útil para futuras mejoras.

Ejemplo:

```
currentAlert = {  
  id: 1,  
  status: "open",  
  title: "Rule matched: Failed login auth"  
};
```

Función `setButtonsForStatus`

```
function setButtonsForStatus(status) {
```

Define una función que recibe el estado actual de la alerta y decide qué botones deben estar habilitados.

Parámetro:

```
status
```

Puede tener uno de estos valores:

```
open  
ack  
closed
```

Seleccionar botón ACK

```
const btnAck = qs("btnAck");
```

Busca el botón con ID:

```
btnAck
```

Este botón está definido en `alert.html`:

```
<button id="btnAck" class="btn primary" type="button">ACK</button>
```

La función `qs()` viene de `app.js`.

Seleccionar botón CLOSE

```
const btnClose = qs("btnClose");
```

Busca el botón de cierre de alerta.

En `alert.html`:

```
<button id="btnClose" class="btn danger" type="button">CLOSE</button>
```

Seleccionar botón REOPEN

```
const btnReopen = qs("btnReopen");
```

Busca el botón para reabrir la alerta.

En `alert.html`:

```
<button id="btnReopen" class="btn" type="button">REOPEN</button>
```

Activar/desactivar ACK

```
btnAck.disabled = (status !== "open");
```

Desactiva el botón ACK si el estado no es `open`.

Esto significa:

```
si status = open    → ACK habilitado
si status = ack     → ACK deshabilitado
si status = closed → ACK deshabilitado
```

Tiene sentido porque solo se reconoce una alerta que está abierta.

Activar/desactivar CLOSE

```
btnClose.disabled = (status === "closed");
```

Desactiva el botón CLOSE si la alerta ya está cerrada.

Esto significa:

```
si status = open    → CLOSE habilitado
si status = ack     → CLOSE habilitado
si status = closed → CLOSE deshabilitado
```

Permite cerrar una alerta abierta o reconocida.

Activar/desactivar REOPEN

```
btnReopen.disabled = (status !== "ack" && status !== "closed");
```

Desactiva REOPEN salvo que la alerta esté en `ack` o `closed`.

Esto significa:

```
si status = open    → REOPEN deshabilitado
si status = ack     → REOPEN habilitado
si status = closed → REOPEN habilitado
```

La lógica es:

```
ack    → puede volver a open
closed → puede volver a open
open   → ya está abierta, no necesita reabrirse
```

Función renderAlert

```
function renderAlert(a) {
```

Define una función que recibe una alerta y la pinta en la página.

Parámetro:

```
a
```

Representa la alerta devuelta por el backend.

Ejemplo de estructura esperada:

```
{
  "id": 1,
  "rule_id": 2,
  "event_id": 15,
  "title": "Rule matched: Failed login auth",
```

```
"group_key": "server-01",
"status": "open",
"created_at": "2026-01-15T12:00:00",
"updated_at": "2026-01-15T12:00:00"
}
```

Guardar alerta actual

```
currentAlert = a;
```

Guarda la alerta recibida en la variable global `currentAlert`.

Esto deja disponible la alerta actual para posibles usos posteriores.

Actualizar título principal

```
qs("title").textContent = `Alerta #${a.id}`;
```

Actualiza el título superior de la tarjeta.

En `alert.html` existe:

```
<h2 id="title">Alerta #</h2>
```

Si `a.id` vale 5, se mostrará:

```
Alerta #5
```

Actualizar subtítulo

```
qs("subtitle").textContent = `status=${a.status} · group_key=${a.group_key ?? "-"}`;
```

Actualiza el subtítulo de la alerta.

Ejemplo:

```
status=open · group_key=server-01
```

El operador:

```
??
```

usa `"-"` si `a.group_key` es `null` o `undefined`.

Esto evita mostrar valores vacíos.

Pintar ID

```
qs("v_id").textContent = a.id;
```

Rellena el campo `id`.

En `alert.html`:

```
<div class="k">id</div><div class="v" id="v_id"></div>
```

Pintar estado

```
qs("v_status").textContent = a.status;
```

Rellena el texto del estado.

Ejemplo:

```
open
```

Clase visual del estado

```
qs("v_status").className = statusBadgeClass(a.status);
```

Actualiza la clase CSS del estado.

`statusBadgeClass()` viene de `app.js`.

Actualmente devuelve siempre:

```
badge
```

Pero esta línea deja preparado el sistema para estilos distintos por estado.

Pintar `group_key`

```
qs("v_group_key").textContent = a.group_key ?? "-";
```

Rellena la clave de agrupación.

Si no existe, muestra:

```
-
```

Pintar `rule_id`

```
qs("v_rule_id").textContent = a.rule_id;
```

Rellena el ID de la regla que generó la alerta.

Este valor permite relacionar la alerta con la tabla `rules`.

Pintar `event_id`

```
qs("v_event_id").textContent = a.event_id;
```

Rellena el ID del evento que disparó la alerta.

Este valor permite relacionar la alerta con la tabla `events`.

Pintar `created_at`

```
qs("v_created_at").textContent = fmtDate(a.created_at);
```

Rellena la fecha de creación.

Usa `fmtDate()` de `app.js` para mostrarla de forma legible.

Pintar `updated_at`

```
qs("v_updated_at").textContent = formatDate(a.updated_at);
```

Rellena la fecha de última actualización.

Este valor cambia cuando se actualiza el estado.

Por ejemplo:

```
open → ack  
ack → closed
```

Pintar título de alerta

```
qs("v_title").textContent = a.title;
```

Rellena el título de la alerta en el panel lateral.

Usa `textContent`, no `innerHTML`, por lo que no interpreta HTML.

Esto es más seguro.

Actualizar botones según estado

```
setButtonsForStatus(a.status);
```

Llama a la función que habilita o deshabilita botones según el estado actual.

Esto evita acciones incoherentes.

Ejemplo:

```
si la alerta está closed:  
  desactivar CLOSE  
  activar REOPEN
```

Función `loadAlert`

```
async function loadAlert() {
```

Define la función que carga una alerta concreta desde el backend.

Es asíncrona porque llama a la API mediante `apiFetch()`.

Seleccionar cajas de mensajes

```
const err = qs("errorBox");  
const info = qs("infoBox");
```

Busca las cajas de error e información definidas en `alert.html`.

Ocultar mensajes anteriores

```
hide(err); hide(info);
```

Ocultar cualquier mensaje previo.

Las funciones `hide()` vienen de `app.js`.

Esto limpia la pantalla antes de cargar la alerta.

Comprobar si falta `alertId`

```
if (!alertId) {  
  show(err, "Falta parámetro ?id= en la URL.");  
  return;  
}
```

Si no existe `alertId`, no se puede consultar ninguna alerta.

Ejemplo de URL incorrecta:

```
alert.html
```

Ejemplo correcto:

```
alert.html?id=1
```

Si falta el parámetro, se muestra el error:

```
Falta parámetro ?id= en la URL.
```

y se sale de la función con `return`.

Bloque `try`

```
try {
```

Inicia un bloque de control de errores.

Si la petición a la API falla, el error se capturará en el `catch`.

Consultar alerta por ID

```
const a = await apiFetch(`/alerts/${alertId}`);
```

Llama al backend para obtener la alerta.

Si `alertId = 1`, la llamada será:

```
GET http://localhost:8000/alerts/1
```

Esto corresponde al endpoint:

```
GET /alerts/{alert_id}
```

del backend.

Renderizar alerta cargada

```
renderAlert(a);
```

Pinta en la página la alerta recibida.

Capturar error de carga

```
} catch (e) {  
  show(err, `Error cargando alerta: ${e.message}`);  
}
```

Si ocurre un error, se muestra en `errorBox`.

Ejemplos:

```
Alert not found  
Failed to fetch  
Update alert failed
```

Función `updateStatus`

```
async function updateStatus(nextStatus) {
```

Define una función para actualizar el estado de la alerta.

Parámetro:

```
nextStatus
```

Representa el nuevo estado que se quiere asignar.

Valores posibles:

```
open  
ack  
closed
```

Seleccionar cajas de mensaje

```
const err = qs("errorBox");  
const info = qs("infoBox");
```

Selecciona las cajas de error e información.

Ocultar mensajes previos

```
hide(err); hide(info);
```

Limpia mensajes anteriores antes de intentar actualizar.

Bloque `try` de actualización

```
try {
```

Inicia el bloque de control de errores de la actualización.

Llamada `PATCH`

```
const a = await apiFetch(`/alerts/${alertId}`, {  
  method: "PATCH",
```

```
body: { status: nextStatus }
});
```

Envía una petición PATCH al backend.

Si `alertId = 1` y `nextStatus = "ack"`, la petición será:

```
PATCH http://localhost:8000/alerts/1
```

Con body:

```
{
  "status": "ack"
}
```

Esto conecta con el backend:

```
PATCH /alerts/{alert_id}
```

que usa el schema:

```
AlertUpdate
```

Método PATCH

```
method: "PATCH",
```

Indica que se quiere modificar parcialmente un recurso existente.

En este caso, solo se modifica el estado de la alerta.

Body de actualización

```
body: { status: nextStatus }
```

Construye el JSON que se enviará al backend.

Ejemplos:

```
{ "status": "ack" }
```

```
{ "status": "closed" }
```

```
{ "status": "open" }
```

Renderizar alerta actualizada

```
renderAlert(a);
```

El backend devuelve la alerta actualizada.

Después se vuelve a pintar toda la vista con los datos nuevos.

Esto actualiza:

```
status
updated_at
```

```
botones habilitados/deshabilitados  
subtítulo
```

Mostrar mensaje de éxito

```
show(info, `Estado actualizado a: ${a.status}`);
```

Muestra un mensaje informativo.

Ejemplo:

```
Estado actualizado a: ack
```

Capturar error de actualización

```
} catch (e) {  
  show(err, `Error actualizando estado: ${e.message}`);  
}
```

Si la actualización falla, se muestra un error.

Posibles causas:

```
alerta no existe  
backend apagado  
estado inválido  
error interno del servidor
```

Función `init`

```
function init() {
```

Define la función de inicialización de la página.

Se ejecuta cuando el DOM está completamente cargado.

Leer ID desde URL

```
alertId = getQueryParam("id");
```

Obtiene el valor del parámetro `id` desde la URL.

`getQueryParam()` viene de `app.js`.

Ejemplo:

```
alert.html?id=5
```

Resultado:

```
alertId = "5";
```

Evento botón ACK

```
qs("btnAck").addEventListener("click", () => updateStatus("ack"));
```

Cuando el usuario pulsa el botón ACK, se llama a:

```
updateStatus("ack")
```

Esto cambia el estado de la alerta a:

```
ack
```

Evento botón CLOSE

```
qs("btnClose").addEventListener("click", () => updateStatus("closed"));
```

Cuando el usuario pulsa CLOSE, se llama a:

```
updateStatus("closed")
```

Esto cambia el estado a:

```
closed
```

Evento botón REOPEN

```
qs("btnReopen").addEventListener("click", () => updateStatus("open"));
```

Cuando el usuario pulsa REOPEN, se llama a:

```
updateStatus("open")
```

Esto reabre la alerta.

Cargar alerta inicial

```
loadAlert();
```

Carga la alerta al abrir la página.

Esto hace que los placeholders – de `alert.html` se sustituyan por datos reales.

Ejecutar cuando el DOM esté cargado

```
document.addEventListener("DOMContentLoaded", init);
```

Espera a que el HTML esté disponible antes de ejecutar `init()`.

Esto es importante porque `init()` busca elementos como:

```
btnAck  
btnClose  
btnReopen
```

Si se ejecutara antes de que existan, fallaría.

7 Relación con el flujo técnico del laboratorio

`alert_detail.js` conecta la vista de detalle con la API de alertas.

Flujo de carga:

```
Usuario abre alert.html?id=1
  ↓
DOMContentLoaded
  ↓
init()
  ↓
getQueryParam("id")
  ↓
loadAlert()
  ↓
GET /alerts/1
  ↓
renderAlert()
  ↓
datos visibles en pantalla
```

Flujo de actualización:

```
Usuario pulsa ACK / CLOSE / REOPEN
  ↓
updateStatus()
  ↓
PATCH /alerts/1
  ↓
backend actualiza Alert.status
  ↓
backend devuelve AlertOut
  ↓
renderAlert()
  ↓
pantalla actualizada
```

8 Relación con backend

Este archivo consume dos endpoints:

```
GET /alerts/{alert_id}
PATCH /alerts/{alert_id}
```

Relación con `alerts.py` del backend:

```
alert_detail.js
  ↓
apiFetch(`/alerts/${alertId}`)
  ↓
get_alert()
  ↓
db.get(Alert, alert_id)
  ↓
AlertOut
```

Para actualizar:

```
alert_detail.js
  ↓
apiFetch(`/alerts/${alertId}`, PATCH)
  ↓
update_alert()
  ↓
AlertUpdate
```

```
↓  
alert.status = payload.status  
↓  
db.commit()  
↓  
AlertOut
```

9 Relación con el flujo SOC

Este archivo implementa la parte interactiva del ciclo de vida de una alerta.

Estados:

```
open  
ack  
closed
```

Acciones:

```
ACK  
↓  
open → ack  
  
CLOSE  
↓  
open → closed  
ack → closed  
  
REOPEN  
↓  
ack → open  
closed → open
```

El frontend no permite cualquier acción en cualquier estado.

La función:

```
setButtonsForStatus(status)
```

controla qué botones están disponibles.

Esto ayuda a mantener un flujo operativo claro.

10 Errores típicos o puntos importantes

La URL necesita ?id=

Sin este parámetro, la página no puede saber qué alerta cargar.

Ejemplo correcto:

```
alert.html?id=1
```

Este archivo usa AlertOut , no AlertUIOut

La llamada actual es:

```
apiFetch(`/alerts/${alertId}`)
```

Por tanto, recibe datos básicos de alerta.

No recibe:


```
rule_name
event_ts
event_source
event_severity
event_message
```

Para mostrar contexto enriquecido, habría que cambiar a:

```
apiFetch(`/alerts/${alertId}/ui`)
```

`currentAlert` está preparado para futuras mejoras

Actualmente se asigna:

```
currentAlert = a;
```

pero no se explota mucho más.

Podría usarse para comparar cambios, mostrar información adicional o evitar recargas innecesarias.

Los botones dependen del estado

La interfaz evita acciones redundantes.

Ejemplo:

```
si la alerta está closed:
  CLOSE queda deshabilitado
  REOPEN queda habilitado
```

`textContent` evita interpretar HTML

Los campos se actualizan con:

```
textContent
```

Esto evita que textos recibidos se interpreten como HTML.

Es más seguro que `innerHTML` para datos dinámicos.

`updated_at` se refresca tras PATCH

Después de actualizar estado, el backend devuelve la alerta actualizada.

El frontend llama a:

```
renderAlert(a)
```

y vuelve a pintar `updated_at`.

`statusBadgeClass()` permite evolución visual

Ahora devuelve una clase neutra, pero permite mejorar el aspecto por estado en el futuro.

Posibles mejoras futuras

Usar endpoint enriquecido

Cambiar:

```
const a = await apiFetch(`/alerts/${alertId}`);
```

por:

```
const a = await apiFetch(`/alerts/${alertId}/ui`);
```

permitiría mostrar:

```
rule_name
event_ts
event_source
event_severity
event_message
```

Añadir contexto del evento

Con `AlertUIOut`, la página podría mostrar un panel como:

```
Evento asociado
├─ source
├─ severity
├─ message
└─ ts
```

Añadir contexto de la regla

También podría mostrar:

```
Regla asociada
└─ rule_name
```

Añadir confirmación al cerrar

Antes de cerrar una alerta, se podría pedir confirmación:

```
¿Seguro que quieres cerrar esta alerta?
```

Añadir notas de investigación

En un flujo SOC más realista, podría añadirse:

```
comentario del analista
motivo de cierre
falso positivo / verdadero positivo
```

1 2 Comandos útiles relacionados

Servir frontend:

```
cd ~/siem-lab/frontend
python3 -m http.server 5173
```

Abrir detalle de alerta:

```
http://localhost:5173/alert.html?id=1
```

Probar endpoint básico usado por la página:

```
curl http://localhost:8000/alerts/1
```

Probar endpoint enriquecido disponible:

```
curl http://localhost:8000/alerts/1/ui
```

Actualizar alerta a `ack` :

```
curl -X PATCH http://localhost:8000/alerts/1 \
-H "Content-Type: application/json" \
-d '{
  "status": "ack"
}'
```

Actualizar alerta a `closed` :

```
curl -X PATCH http://localhost:8000/alerts/1 \
-H "Content-Type: application/json" \
-d '{
  "status": "closed"
}'
```

Reabrir alerta:

```
curl -X PATCH http://localhost:8000/alerts/1 \
-H "Content-Type: application/json" \
-d '{
  "status": "open"
}'
```

Abrir consola del navegador:

```
F12 → Console
```

Errores habituales que podrían aparecer:

```
Falta parámetro ?id= en la URL.
Error cargando alerta: Alert not found
Error actualizando estado: ...
Failed to fetch
```